esprit Se former autrement HONORIS UNITED UNIVERSITIES	**Examen** **Semestre : 1** **Session : PRINCIPALE**
Module : Système d'exploitation avancé (SEA1) **Enseignant(s)** : UP Système **Classe(s) :** 4EME ANNEE	**Documents autorisés** : NON **Nombre de pages** : 2 **Date** : 09/11/2022 **Heure** 13h00 **Durée** : 01h30

Exercice 1 (8points)

Partie 1 (4points)

Ecrire un programme qui crée **2 processus fils**, l'un exécute la commande **ls -l**, l'autre **ps.** Le père devra attendre la fin de ses deux fils puis affiche le PID du premier processus terminé.

```
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
int main(int argc, char * argv[]) {
pid_t pid1,pid2,pid_premier;
int status;
switch(pid1=fork()) {
case -1:
perror("fork error");

break;
case 0:
execlp("ls","ls", "-l",NULL);
break;
default:
switch(pid2=fork()) {
case -1:
perror("fork error");
break;
case 0:
execlp("ps","ps",(char *) 0);
break;
default:
break;
}
break;

}

pid_premier = wait(&status);

wait(&status);

printf("Premier processus a finir : %d\n", (pid_premier==pid1)?1:2);}
```

Partie 2 (4points)

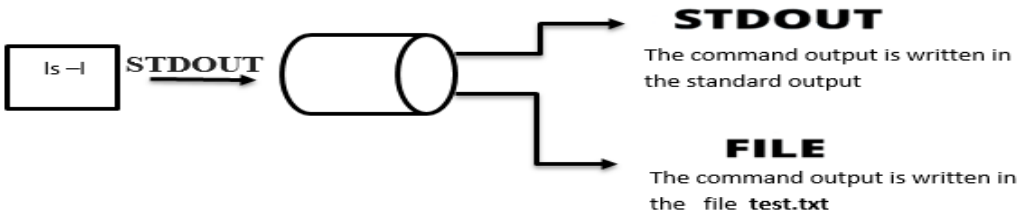
Ecrire un programme **log** qui permet d'exécuter une commande quelconque en récupérant automatiquement une copie de sa sortie standard dans un fichier. En quelque sorte, il s'agit de dédoubler chacun des caractères envoyés sur sa sortie standard : un exemplaire est reproduit sur la sortie standard par défaut alors que l'autre est enregistré dans un fichier.

Le programme log prend en premier argument le nom du fichier à utiliser pour la sortie. Les arguments suivants constituent la commande à exécuter.
Pour réaliser le dédoublement de la sortie standard d'un processus, une solution est de rediriger sa sortie standard vers un tube à l'extrémité duquel un second processus pourra alors facilement extraire tous les caractères et les sauver dans un fichier en même temps qu'il les affichera sur sa sortie standard.

Le programme log va donc créer un processus fils et un tube partagé par le père et le fils. Le fils, par exemple, pourra exécuter la commande spécifiée en paramètre, tandis que le père se chargera de l'affichage et de l'écriture dans le fichier.

Donnez le code complet du programme log en utilisant autant que possible des fonctions d'entrée/sortie de haut niveau.

Exemple d'exécution : «. /log test.txt ls –l »



Annexe : un programme qui permet d'écrire bonjour dans un fichier **file.txt**
int file_desc = **open**("file .txt", O_WRONLY | O_APPEND) ;
write(file_desc, "bonjour ",7);
avec file_desc est le descripteur de fichier file.txt.

```
include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <sys/wait.h>
/* log <file> <command> [ <args> ]
 *
 * Exécution de la commande "<command> <args>" et on duplique sa
 * sortie standard dans le fichier <file>.
 *
 * Exemple : gcc -std=c99 log.c ; rm toto ; ./a.out ./toto ls
 */
int main (int argc, char *argv[])
{
  int pid, p[2], file;

  // création d'un pipe partagé par le père et le fils
  if(pipe(p) == -1)
    { perror("pipe"); exit(EXIT_FAILURE); }

  /*
   *              --------------------------
   * Fils: 1 ---> ( p[1]      PIPE      p[0] ( ---> Fils: <file>
   *              --------------------------
```

```
 *
 */

  // création d'un processus fils...
  if((pid = fork()) == -1)
    { perror("fork"); exit(EXIT_FAILURE); }

  // le processus fils...
  // ... éxécute la commande en redirigeant sa sortie standard dans le
pipe
  if(pid == 0) {
    dup2(p[1],1); // redirection de la sortie standard
    close(p[0]); close(p[1]);  // ne pas oublier les fermetures
    execvp(argv[2],argv+2); // <command> remplace le processus fils
    perror("execvp");
  }

  // le père...
  // ... lit l'entrée du pipe, l'affiche et l'écrit dans un fichier
  file = open(argv[1], O_WRONLY|O_CREAT|O_TRUNC,
S_IRUSR|S_IWUSR);
  if(file == -1)
    { perror("open file"); exit(EXIT_FAILURE); }
  close(p[1]);

  char c;
  // Nota Bene : lecture bloquante sur tube vide (je m'endors) !
  while (read(p[0],&c,1) == 1) { // renvoie 0 s'il n'y a plus d'écrivain
    write(1,&c,1); // affichage sur la sortie standard
    write(file,&c,1); // écriture dans le fichier
  }

  // ne pas oublier la fermeture des descripteurs restants...
  close(file); close(p[0]);
```

```
  // j'attend la fin de mon fils...
  wait(NULL);

  return EXIT_SUCCESS;
}
```

Exercice 2 (5 points)

1. donner l'arborescence générée par P1 et P2

P1	P2

int main (void)
{
int i,n= 3;
int fils_pid ;
for (i = 1 ; i < n ; i++)
{
fils_pid = fork ( ) ;
if (fils_pid > 0) break ;
 	
printf (" Processus %d avec père %d , i=%d\n" , getpid ( ) ,
getppid ( ) , i) ;
 	
}
}
	int main (void)
	{
	fork ( ) ;
	fork ( ) ;
	fork ( ) ;
	fork() ;
	}

Exercice 3 (6points)

Soit les différents processus suivants :

Processus	Temps d'arrivée	Temps d'exécution	Priorité
A	0	5	4
B	2	4	2
C	2	2	6
D	4	4	3

Remarque : la priorité est croissante. (i+1)> i

Temps de séjour =Temps de rotation= Temps de traitement

1. Donner le diagramme de GANTT illustrant l'ordonnancement des processus en utilisant l'algorithme avec **priorité non préemptif** et calculer le temps moyen de séjour.
2. Donner le diagramme de GANTT illustrant l'ordonnancement des processus en utilisant l'algorithme avec **priorité préemptif** et calculer le temps moyen de séjour.
3. Donner le diagramme de GANTT illustrant l'ordonnancement des processus en utilisant l'algorithme avec priorité non préemptif et un quantum **q=3 (hybridation entre l'algorithme RR et priorité non préemptif)** ;